

Pętle i tablice

1. Napisać program, który policzy sumę liczb od 1 do n , n (liczbę całkowitą) ma podać użytkownik. (*pętla for*)
2. Napisać program, który wyświetli na ekranie "schodki" z n piętrami, n podaje użytkownik, np dla $n = 5$ (*pętla for zagnieżdżona*):

```
#
##
###
####
#####
```

3. Wypełnić tabelę 2-wymiarową `int tab[10][10]` wynikami mnożenia liczb od 1 do 10, `tab[i][j]` ma być równe $(i+1)*(j+1)$, gdzie $i = 0, 1 \dots 9$ i tak samo $j = 0, 1, 2 \dots 9$. Wyświetlić na ekranie zawartość tablicy, każdy i -ty wiersz ma się znaleźć w odrębnej linii (*pętla for zagnieżdżona i tablica int'ów*).

1. **Tip:** drobna modyfikacja specyfikatora formatu dla `int` spowoduje, że dana liczba będzie wyświetlana zawsze w "polu" o długości np. 4, tj. jeśli napiszemy `printf("%4d",x)` zamiast `printf("%d",x)` a `x=20`, to wyświetli nam się:

- 20
- a nie po prostu `x`
- tak samo, jak `x=4`, zobaczymy `x`,
- w przypadku `x=111`, `111` etc.

4. Wypełnić tablicę 4×4 kolejno liczbami od 1 do $4 \times 4 = 16$. **UWAGA:** powinno działać dla dowolnego rozmiaru tablicy kwadratowej ≥ 2 , tutaj robimy na 4×4 , bo łatwo wyświetlić przykłady dla tej wielkości w ramach tego dokumentu. *Na razie możemy wpisać w kodzie rozmiar tablicy "na sztywno", albo dodać `#define N 4` na początku pliku.*

- Wyświetlić ją w porządku standardowym:

```
1  2  3  4
5  6  7  8
9 10 11 12
13 14 15 16
```

- o wyświetlić spłaszczoną tablicę w porządku "row-major", a potem "column-major":

```
row major:  1  2  3  4  5  6  7  8  9 10 11 12 13 14 15 16

column major:  1  5  9 13  2  6 10 14  3  7 11 15  4  8 12 16
```

- o wyświetlić kolejne wiersze w naprzemiennej kolejności:

```
alternating rows:  1  2  3  4  8  7  6  5  9 10 11 12 16 15 14 13
```

- o wyświetlić kolejne "ramki": 1 wiersz i 1 kolumna, potem 2 wiersz i 2 kolumna (ale bez ostatniego elementu, który był w ramce nr 1), potem 3 wiersz i 3 kolumna (bez 2 ostatnich, które już wypisano)...itd. :

```
frames:  1  2  3  4  8 12 16  5  6  7 11 15  9 10 14 13
```

- o (*) wyświetlić po kolei przekątne tablicy, biegnące z prawej do lewej, każdą z przekątnych po kolei, najpierw uporządkowaną w jednym możliwym porządku, potem drugim:

```
RL diagonals:  1  2  5  3  6  9  4  7 10 13  8 11 14 12 15 16

reversed:  1  5  2  9  6  3 13 10  7  4 14 11  8 15 12 16
```

- o (*) tak samo jak wyżej, tylko przekątne biegnące z lewej do prawej:

```
LR diagonals: 13 14  9 15 10  5 16 11  6  1 12  7  2  8  3  4

reversed: 13  9 14  5 10 15  1  6 11 16  2  7 12  3  8  4
```

5. Pobrać od użytkownika jego imię do tablicy `char imie[80]` oraz w tablicy `char imie_reverse[80]`, umieścić litery tego imienia w odwrotnej kolejności. Wyświetlić na ekranie imię po odwróceniu, np. `Robert -> treboR`. (można za pomocą `for + break`, albo `while`. w tablicy `imie`, trzeba wykryć pierwszą pozycję znaku `'\0'`, który oznacza koniec łańcucha znaków. Wykorzystać to, aby do tablicy `imie_reverse` zapisać te znaki w odwróconej kolejności. Po wpisaniu wszystkich liter w `imie_reverse`, powinno się znowu wstawić znak `'\0'`, oznaczający koniec linijki) ...**mamy oczywiście wbudowaną funkcję `strlen`, która nam zwróci pozycję tego znaku, ale dobrze to spróbować zrobić samemu- aby poćwiczyć szukanie elementu w tablicy.**

6. Wypełnić elementy 2-wymiarowej tablicy znaków `char chessboard[8][16]` naprzemiennie ciągami znaków `"__"` oraz `"###"`, aby po wyświetleniu jej linijka-po-linijce, uzyskać poniższy wynik:

```
__##_##_##_##_##_
##_##_##_##_##_
__##_##_##_##_##_
##_##_##_##_##_
__##_##_##_##_##_
##_##_##_##_##_
__##_##_##_##_##_
##_##_##_##_##_
```

(zagnieżdżony `for`. w wewnętrznej pętli, zwracamy uwagę albo na reszty z dzielenia przez 4 zmiennej po której iterujemy, albo radzimy sobie inaczej - robiąc kroki w pętli o długości 2 lub 4...)

7. Napisać program, który pobierze od użytkownika liczbę rzeczywistą m . Potem będzie dzielił ją przez 2, aż uzyskamy wartość mniejszą niż 1. Program ma zawsze wykonać co najmniej jedno dzielenie. Po każdym dzieleniu, ma się wyświetlić na ekranie jego wynik (pętla `do while` lub `while`. Da się nawet używając `for(int i=0; i< k; i++)`, tylko trzeba użyć logarytmu aby ustalić k - ile razy można podzielić m przez 2).